Project Milestone 1 Report: Classical Planning and Control Foundation for Planner-Guided Robotic Manipulation

Shrikrishna Rajule (039291041)

***Abstract*—This report presents the Milestone 1 implementation of a classical planning and control foundation for a hybrid robotic manipulation project. The target long-term system combines configuration-space motion planning with residual reinforcement learning for robust tabletop manipulation. In this milestone, the focus is restricted to the classical backbone required before learning-based adaptation can be introduced. A Franka Emika Panda 7-DoF manipulator is simulated in PyBullet in a tabletop scene with a cube object. The implemented system includes geometry-aware scene setup, robot and environment state extraction, inverse-kinematics-based task decomposition, collision-aware joint-space planning with RRT*, waypoint execution, a custom joint-space PD controller, grasp validation using contact and geometric checks, attached-object handling during transport, and nominal pick-and-place execution. The final system successfully performs a full pick-and-place sequence including pre-grasp, descend, grasp, lift, transfer, placement, retreat, and return-to-home. This milestone therefore satisfies the course requirement of establishing a simulation environment, loading the robot, obtaining environment and robot state information, and implementing a classical component for the project.**

***Index Terms*—Robotic manipulation, motion planning, RRT*, PyBullet, Panda robot, PD control, pick-and-place, classical robotics.**

I. Introduction

This project studies a hybrid robotic manipulation framework that combines structured classical motion planning with learning-based residual adaptation for robust tabletop task execution. The long-term objective is to preserve the feasibility, structure, and interpretability of geometric planning while later introducing reinforcement learning to compensate for modeling errors, uncertainty, and execution mismatch. The manipulation platform selected for this project is the Franka Emika Panda 7-DoF manipulator simulated in PyBullet, with a tabletop cube manipulation task used as the main experimental setting.

The project proposal defined three major stages. Milestone 1 focuses on the planning and control foundation: loading the Panda model in PyBullet, implementing joint/configuration-space RRT* with collision checking, generating executable trajectories, implementing PD-based tracking control, and demonstrating nominal pick-and-place execution. Milestone 2 is intended to introduce controlled perturbations and residual PPO-based adaptation, while Milestone 3 extends evaluation toward robustness experiments and optional additional manipulation tasks. The current report therefore addresses the first milestone only, with emphasis on a classical manipulation pipeline that can later support learning-based residual control.

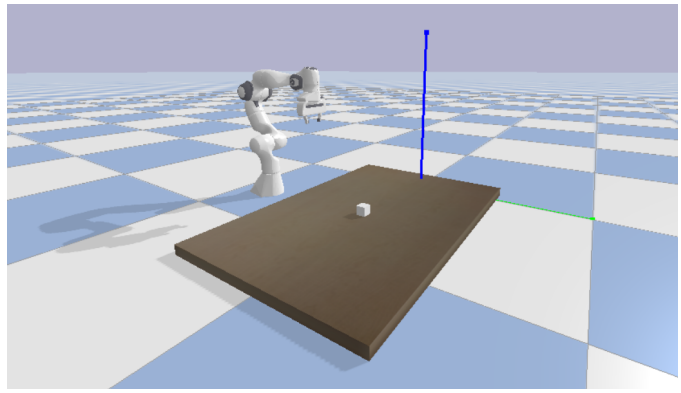


Fig. 1. PyBullet simulation scene used in Milestone 1. The Franka Panda is placed beside the table, and the cube object is initialized on the tabletop before the grasp sequence begins.

The implementation developed in this milestone goes beyond a simple joint-space motion demonstration. The final system includes a complete nominal pick-and-place routine: scene setup, geometry-aware object placement, robot/environment state extraction, joint-space planning, inverse-kinematics-based Cartesian subgoals, custom PD waypoint tracking, contact-gated grasp validation, attached-object transport, placement, retreat, and return-home. This makes the milestone not only a planning benchmark, but also a full classical nominal manipulation baseline. It establishes the nominal controller that later learning methods will build upon. This report documents that classical backbone.

II. Methods

A. Simulation Environment Setup

The simulation environment is implemented in PyBullet and includes four main components: a ground plane, a table, a cube object, and a fixed-base Franka Emika Panda manipulator. PyBullet was chosen because it provides URDF-based robot loading, rigid-body simulation, collision checking, kinematic queries, and rapid prototyping support for both classical robotics and reinforcement learning workflows.

A geometry-aware scene initialization procedure is used to make the environment reproducible. After loading the table model, the table axis-aligned bounding box is queried and the table is translated vertically so that its bottom rests on the plane. The tabletop height is then computed from the table AABB. The cube is placed on the tabletop by aligning its

bottom with the measured table top height and applying a small clearance. This approach avoids relying on fragile hard-coded support heights and makes the setup less sensitive to mesh origin assumptions.

The Panda is placed beside the table and oriented toward the workspace. During implementation, the robot base position was tuned iteratively because reachability strongly affected grasp quality and placement behavior. A more distant base placement caused stretched-arm postures and grasp alignment problems. Moving the base closer produced a significantly more natural reachable workspace and allowed the final grasp routine to succeed without persistent manual Y-axis bias.

B. Software Organization

The codebase is organized into modular components so that planning, control, task logic, and evaluation can evolve independently. The key modules are:

- `controller/`: custom PD control implementation,
- `demo/`: initial development, and task-level pick-and-place sequencing implementations,
- `evaluations/`: scripts to evaluate and plot runs,
- `planner/`: collision-aware joint-space planning, including RRT and RRT*,
- `sim/`: environment loading, robot interface, and state extraction,
- `trajectory/`: interpolation of sparse paths into executable waypoint sequences,
- `utils/`: utility modules, TrajectoryLogger,
- `config.py`: experiment configuration parameters,
- `main.py`: experiment entry point.

C. Robot Loading and State Extraction

The Panda URDF is loaded as a fixed-base manipulator. Joint metadata is queried from the simulator to extract the 7 arm joints, 2 gripper joints, joint limits, and the end-effector/grasp-target link index. The simulator is also queried for the robot's joint positions, joint velocities, end-effector pose, cube pose, and the table AABB.

These state queries serve multiple purposes. They are used for debugging initialization, validating scene geometry, computing task-space targets, checking tracking performance, logging motion execution, and validating grasp/contact conditions.

D. Classical Motion Planning

The core classical planning component is joint/configuration-space motion planning with collision checking. The planner first attempts direct collision-free interpolation between start and goal. If a direct edge is infeasible, the system uses a sampling-based tree planner. The final implementation supports both RRT and RRT*, with RRT* used as the primary planner because it improves path quality through rewiring and better parent selection.

Planner behavior includes:

- joint-limit checking,
- collision-free configuration validation,
- collision-free edge checking via interpolation,
- optional path shortcut smoothing,
- interpolation of sparse planner paths into dense execution waypoints.

An important refinement made during the pick-and-place implementation was contact-aware planning for attached-object phases. Early versions of the planner treated all contacts too conservatively and frequently failed during lift, transfer, or place phases when the robot was holding the cube. This caused the system to fall back to coarse interpolation, which led to visibly jerky motion. The planner was later improved by filtering allowed contacts during attached-object motion, making the planning pipeline more consistent and reducing unnecessary fallback behavior.

E. Custom Joint-Space PD Control

A custom joint-space PD controller is used for waypoint tracking instead of relying entirely on the simulator's default position servo. The control law computes desired joint velocity commands from joint position and velocity error:

$$\dot{q}_{cmd} = K_p(q_{des} - q) + K_d(\dot{q}_{des} - \dot{q}). \tag{1}$$

These velocity commands are then applied through PyBullet's velocity control interface. This provides a clearer separation between:

- path generation,
- tracking control,
- simulator actuation.

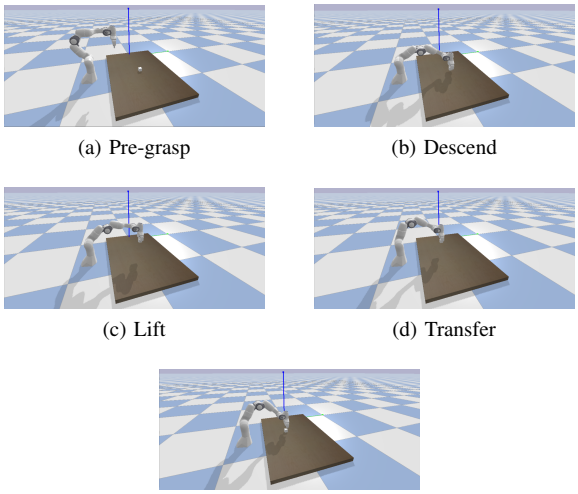
The controller gains, velocity limits, and waypoint tolerances were tuned iteratively to balance speed, smoothness, and convergence. During development, excessive aggressiveness caused failures to converge at strict waypoint tolerances, while overly conservative values produced very slow motion. The final settings provide a stable compromise for the nominal pick-and-place task.

F. Pick-and-Place Task Pipeline

To satisfy the nominal manipulation objective from the proposal, the project was extended from generic planning to a full classical pick-and-place sequence. The task is decomposed into the following stages:

1) home,
2) pre-grasp,
3) grasp descend,
4) close gripper,
5) validate grasp and attach object,
6) lift,
7) transfer to pre-place,
8) place descend,
9) release object,
10) open gripper,
11) retreat,
12) return-home.

Cartesian subgoals for pre-grasp, grasp, lift, pre-place, place, and retreat are generated from the measured cube pose and table height. These task-space subgoals are converted into



(a) Pre-grasp (b) Descend (c) Lift (d) Transfer (e) Place

Fig. 2. Sequence frames from the nominal pick-and-place execution. The robot moves from pre-grasp to descend, lifts the cube, transfers it to the target region, and places it on the table.

joint targets using inverse kinematics, then passed through the same joint-space planner and execution stack as any other motion command.

G. Grasp Validation and Object Handling

A major debugging issue early in development was that the cube could appear to move even when the robot had not clearly achieved a physical grasp. This happened because initial versions attached the cube after closing the gripper without verifying that the hand was actually close enough or in contact with the object. That behavior was corrected by introducing grasp validation before attachment.

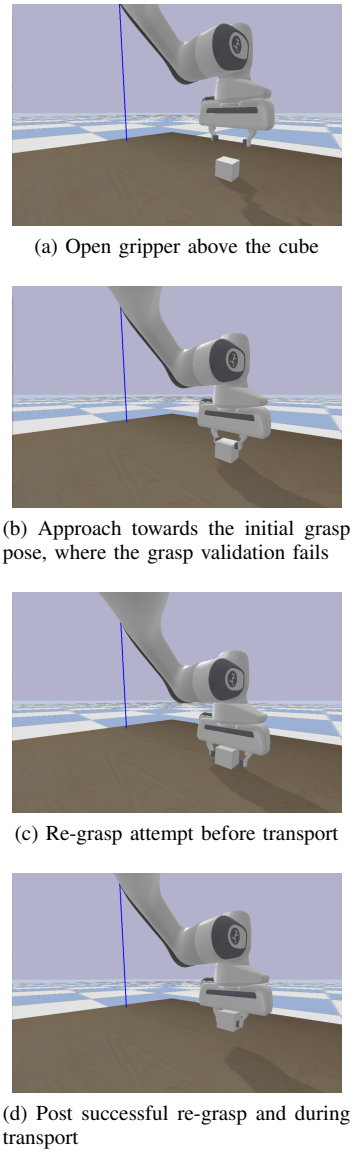
The final grasp validation logic checks:

- end-effector-to-cube distance,
- finger-to-cube distance,
- total contact count,
- finger contact count.

Only when the grasp is marked ready is the cube attached to the robot with a fixed constraint for stable transport. If the initial descend does not satisfy the grasp criteria, the system performs a deeper retry descend and evaluates again. This retry logic proved important in the final system and is one reason the nominal task is robust enough to complete consistently.

H. Placement Refinement

Placement accuracy also required refinement. Earlier versions used explicit snapping of the cube to the place target before release, which improved apparent placement accuracy but caused visually odd behavior if the robot had not physically reached the exact target pose. In the improved configuration, the place target itself was moved to a more reachable region toward the robot, reducing stretch and making the final placement more natural. This change was preferable to using a large manual grasp or place bias and better reflected the project goal of physically plausible nominal execution.



(a) Open gripper above the cube

(b) Approach towards the initial grasp pose, where the grasp validation fails

(c) Re-grasp attempt before transport

(d) Post successful re-grasp and during transport

Fig. 3. Grasp validation and object handling sequence. The robot approaches the cube, closes the gripper, validates the grasp based on proximity and contact conditions, and only then attaches the cube for transport.

I. Logging and Evaluation Support

The execution pipeline logs task phases, waypoint indices, simulation steps, target joint positions, actual joint positions, joint errors, and end-effector pose information. These logs were crucial for diagnosing:

- grasp undershoot and overshoot,
- stretched-arm reachability problems,
- jerky fallback execution,
- placement offset,
- residual return-home error.

The final implementation therefore not only executes the task, but also exposes sufficient metrics for later evaluation, comparison, and future learning experiments.

III. Preliminary Experimental Results

The final Milestone 1 system successfully demonstrates a complete nominal pick-and-place routine in simulation. The

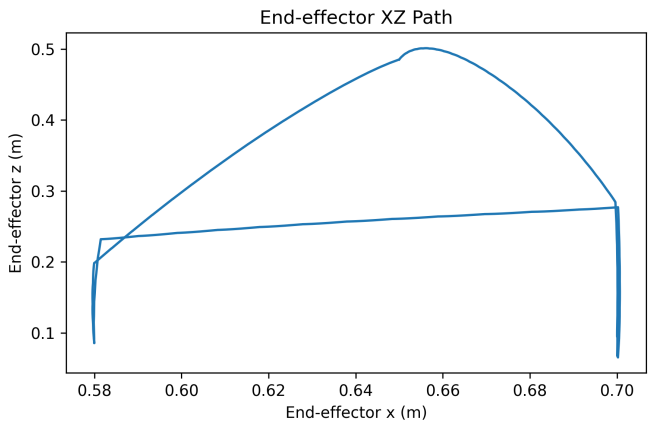


Fig. 4. End-effector trajectory projected into the x-z plane during the pick-and-place execution. The trajectory reflects the nominal pre-grasp, descend, lift, transfer, and placement phases.

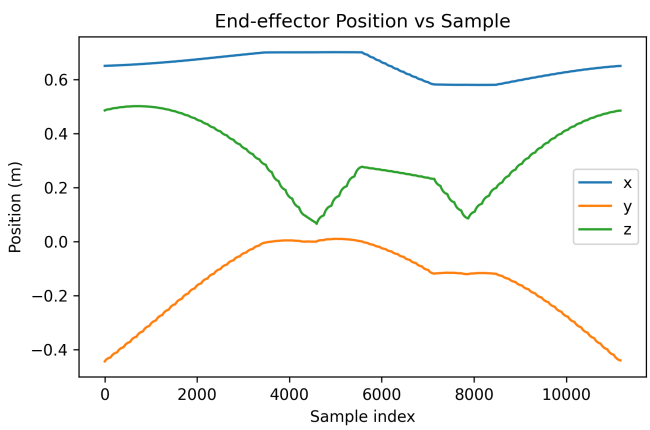


Fig. 5. End-effector Cartesian position versus sample index during the final nominal pick-and-place execution.

observed sequence is:

1) the Panda initializes in a stable home configuration beside the table,
2) the robot moves above the cube,
3) the end effector descends toward the object,
4) the gripper closes,
5) the system validates the grasp using contact and geometric checks,
6) the cube is lifted, transported, and placed,
7) the robot retreats and returns home.

This is a substantial progression beyond the early blocked-case planning benchmark, which primarily demonstrated collision-aware joint-space motion generation and execution.

Qualitatively, several implementation lessons emerged during the milestone:

- **Workspace geometry matters strongly.** When the robot base was too far from the table, the arm became stretched and the grasp target became difficult to reach reliably.
- **Grasp success must be validated explicitly.** Unconditional object attachment can create false success signals in simulation.

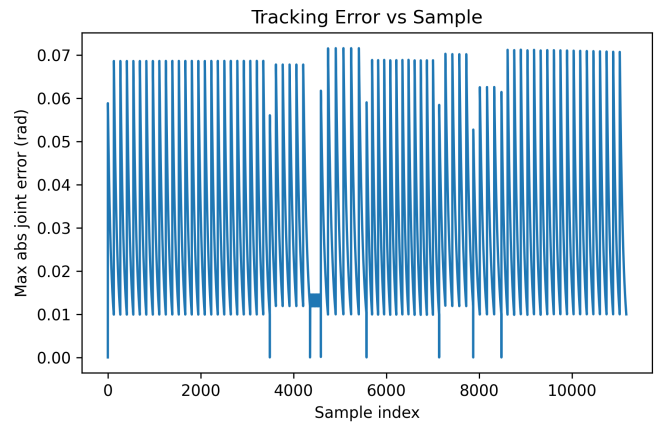


Fig. 6. Maximum absolute joint tracking error versus sample index during the final pick-and-place execution. This plot reflects the performance of the custom PD waypoint tracking controller.

- **Attached-object motion needs contact-aware planning.** Otherwise the planner may incorrectly reject feasible states during lift or place phases and fall back to coarse interpolation.
- **Reachable place targets matter.** A place target that is too far away can degrade placement accuracy and create visually unrealistic behavior.

The final system now behaves consistently enough to be treated as a valid nominal baseline. It achieves the main Milestone 1 goal: an operational classical simulation pipeline for manipulation using planning and control.

The experiment demonstrates several meaningful outcomes:

- successful simulation setup and robot loading,
- valid state extraction from both robot and environment,
- collision-aware planner usage in a manipulation context,
- stable custom PD tracking,
- nominal pick-and-place execution in a tabletop task.

This baseline is sufficiently complete to support future robustness studies and learning-based extension.

IV. Future Work

The next milestone is to move from nominal classical execution to robust execution under uncertainty. As outlined in the proposal, the next step is to introduce controlled perturbations in the object pose and/or execution process and then augment the nominal classical controller with residual reinforcement learning, primarily using PPO.

The future work can be grouped into four directions.

A. Residual Reinforcement Learning

The main learning objective is to add a bounded residual policy on top of the classical planner-controller stack. Conceptually, the project aims to study a hybrid structure of the form

$$\tau_{total} = \tau_{PD} + \tau_{RL}, \tag{2}$$

where the planner/controller produces nominal structured behavior and the learned residual compensates for execution

mismatch or uncertainty. Even if the simulator implementation uses velocity-level control in the current milestone, the hybrid control idea remains unchanged.

B. Robustness Evaluation

The current milestone demonstrates only nominal pick-and-place. Future experiments should introduce perturbations such as:

- randomized object pose offsets,
- execution noise,
- varied placement goals,
- repeated trials for success-rate estimation.

These experiments will make it possible to compare planner-only performance against planner-guided residual RL and, later, RL-only baselines.

C. Stronger Quantitative Metrics

The current logging system already provides a strong basis for quantitative analysis. Future evaluation metrics should include:

- pick success rate,
- grasp success rate before and after retry descend,
- placement error relative to the place target,
- trajectory length and planner cost,
- simulation-step count,
- final tracking error,
- success under increasing perturbation magnitude.

D. Extended Manipulation Tasks

The proposal also lists optional extensions such as drawer opening, SAC comparison, and peg-in-hole evaluation. These are not required to satisfy the current project promise, but they provide a path for deeper study if the core planner-guided residual PPO system is completed early.

V. Conclusion

This milestone established the classical planning and control foundation for the proposed planner-guided robotic manipulation project. A Franka Panda manipulator was loaded into PyBullet, a reproducible tabletop environment was constructed, robot and environment state information were extracted, collision-aware joint-space planning was implemented using RRT*, a custom PD controller was used for tracking, and the resulting system was extended into a full nominal pick-and-place routine.

The final implementation satisfies the Milestone 1 expectations from both the course and the project proposal. More importantly, it provides the classical backbone needed for the remainder of the project. With a working nominal manipulation pipeline now in place, the project is well-positioned to advance toward robustness evaluation and planner-guided residual reinforcement learning in the next milestones.

Acknowledgment

The author acknowledges the CS558 course project framework and the guidance provided through the milestone requirements, which helped structure this work around a clear progression from classical planning and control to future learning-based manipulation.

References

[1] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016–2025. [Online]. Available: http://pybullet.org

[2] Franka Emika, "Franka Emika Panda," [Online]. Available: https://www.franka.de